



Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

System Ekspertowy do Identyfikacji Gatunków Zwierząt

Dokumentacja Końcowa

Autorzy:

Krzysztof Gryboś
Michał Piechota
Sebastian Rydz
Hubert Sobociński

Przedmiot: Systemy Ekspertowe
Rok akademicki 2025/2026

Warszawa, 2026

Spis treści

1	Wprowadzenie i Cel Projektu	2
1.1	Zakres funkcjonalny	2
1.2	Struktura projektu	2
2	Modelowanie Wiedzy	2
2.1	Reprezentacja atrybutów rozmytych	3
2.1.1	Typy funkcji przynależności	3
2.1.2	Atrybuty rozmyte w systemie	4
2.2	Pozostałe atrybuty	4
2.3	Implementacja bazy wiedzy	4
3	Mechanizm Wnioskowania i Interakcja z Użytkownikiem	5
3.1	Dynamiczna selekcja pytań	5
3.2	Kontekstowość dialogu	6
3.3	Wielokrotny wybór	6
4	Moduł wnioskowania rozmytego	6
4.1	Przykład liczbowy	7
5	Moduł wnioskowania przybliżonego	7
5.1	Relacja nierozróżnialności	8
5.2	Przybliżenie dolne i górne	8
5.3	Niespójność	8
5.4	Atrybuty niezbędne i jądro systemu	8
5.5	Redukty	8
5.5.1	Wyznaczanie reduktów z macierzy nierozróżnialności	9
5.6	Reguły minimalne	9
5.7	Przykład: wyznaczenie reguły minimalnej	9
6	Architektura Systemu – Separacja Warstw	10
6.1	Komponenty systemu	10
6.2	Wymagania dotyczące separacji	10
7	Instalacja i uruchomienie	11
7.1	Rozszerzanie bazy wiedzy	11
7.2	Analiza bazy wiedzy	11
8	Przykładowa sesja identyfikacji	12
9	Testy i Weryfikacja	12
10	Ograniczenia i Kierunki Rozwoju	13
11	Harmonogram Realizacji	13
12	Podział Obowiązków	13
13	Podsumowanie	13

1 Wprowadzenie i Cel Projektu

Niniejszy dokument stanowi dokumentację końcową projektu zrealizowanego w ramach przedmiotu *Systemy Ekspertowe*. Opisuje on gotowy, działający system ekspertowy, którego zadaniem jest identyfikacja gatunku zwierzęcia na podstawie jego cech fenotypowych (takich jak masa ciała, długość, długość życia, pokrycie ciała) oraz środowiskowych (np. środowisko życia, kontynent występowania).

W odróżnieniu od prostych algorytmów decyzyjnych opartych na drzewach decyzyjnych, zaimplementowany system wykorzystuje **wnioskowanie dedukcyjne w środowisku niepewnym**. Łączy on ścisłą logikę języka Prolog z mechanizmami logiki rozmytej (ang. *fuzzy logic*) oraz teorią zbiorów przybliżonych (ang. *rough sets*), co pozwala na obsługę parametrów o charakterze ciągłym i prowadzenie wnioskowania przybliżonego.

Całość systemu została zaimplementowana w języku **SWI-Prolog**. Pomocniczy skrypt w języku Python posłużył do wygenerowania bazy wiedzy z pliku danych w formacie CSV.

1.1 Zakres funkcjonalny

Zaimplementowany system umożliwia:

- klasyfikację zwierząt na podstawie cech podawanych interaktywnie przez użytkownika,
- wnioskowanie z niepewnością przy użyciu funkcji przynależności zbiorów rozmytych,
- dynamiczne sterowanie dialogiem w oparciu o moc dyskryminacyjną atrybutów (mierzoną entropią),
- analizę bazy wiedzy pod kątem reguł minimalnych, atrybutów niezbędnych (jądra) oraz reduktów.

1.2 Struktura projektu

Projekt został podzielony na pliki odpowiadające poszczególnym warstwom systemu. Strukturę plików przedstawiono w tabeli 1.

Tabela 1: Struktura plików projektu

Plik	Opis
main.pl	Punkt wejścia. Menu wyboru: uruchomienie systemu lub analiza bazy wiedzy.
src/kb.pl	Baza wiedzy – fakty o 100 gatunkach (atrybuty dyskretne i ciągłe).
src/fuzzy.pl	Moduł wnioskowania rozmytego (funkcje przynależności, t-norma, wyznaczanie atrybutu <i>wielkość</i>).
src/rough.pl	Moduł wnioskowania przybliżonego (relacja nierozróżnialności, redukty, reguły minimalne, klasyfikacja).
src/ui.pl	Interfejs użytkownika – dynamiczny dialog i selekcja pytań.
gen_kb.py	Skrypt generujący kb.pl z pliku <i>zwierzeta_atrybuty.csv</i> .
zwierzeta_atrybuty.csv	Dane źródłowe: 100 gatunków wraz z wartościami atrybutów.

2 Modelowanie Wiedzy

Baza wiedzy nie jest prostym wykazem gatunków, lecz zbiorem faktów oddzielających wiedzę o dziedzinie od logiki sterującej. Każde zwierzę opisane jest zestawem atrybutów dyskretnych (deklarowanych predykatem `attribute/3`) oraz ciągłych (predykat `continuous_attribute/3`).

2.1 Reprezentacja atrybutów rozmytych

Jednym z kluczowych wyzwań jest obsługa parametrów rozmytych, takich jak masa lub długość ciała. Zamiast stosować sztywne progi binarne przynależności (np. `masa > 100 kg`), system wykorzystuje teorię zbiorów rozmytych. Zdefiniowane są funkcje przynależności $\mu_A(x)$, co umożliwia przypisanie cechy do obiektu z częściową pewnością.

2.1.1 Typy funkcji przynależności

Funkcja trapezoidalna Stosowana dla atrybutów posiadających przedział pełnej przynależności i częściową przynależność na obu jego końcach. Oznaczamy ją jako $\text{Trapez}(a, b, c, d)$.

$$\mu_T(x; a, b, c, d) = \begin{cases} 0 & x \leq a \\ \frac{x-a}{b-a} & a < x \leq b \\ 1 & b < x \leq c \\ \frac{d-x}{d-c} & c < x \leq d \\ 0 & d < x \end{cases} \quad (1)$$

Funkcje L i R-kształtne Stosowane dla atrybutów, które dla wartości mniejszych (większych) od wartości granicznej mają pełną przynależność, a na krańcu przynależność częściową. Oznaczamy je jako $\text{L-kształtna}(a, b)$ oraz $\text{R-kształtna}(a, b)$.

$$\mu_L(x; a, b) = \begin{cases} 1 & x \leq a \\ \frac{b-x}{b-a} & a < x \leq b \\ 0 & b < x \end{cases} \quad \mu_R(x; a, b) = \begin{cases} 0 & x \leq a \\ \frac{x-a}{b-a} & a < x \leq b \\ 1 & b < x \end{cases} \quad (2)$$

W implementacji funkcje te zostały zrealizowane bezpośrednio w pliku `fuzzy.pl`:

Listing 1: Funkcje przynależności (`fuzzy.pl`)

```
trapez(X, A, B, C, D, Val) :-
    ( X =< A -> Val = 0.0
    ; X > A, X =< B -> Val is (X - A) / (B - A)
    ; X > B, X =< C -> Val = 1.0
    ; X > C, X =< D -> Val is (D - X) / (D - C)
    ; Val = 0.0
    ).

l_ksztaltna(X, A, B, Val) :-
    ( X =< A -> Val = 1.0
    ; X > A, X =< B -> Val is (B - X) / (B - A)
    ; Val = 0.0
    ).

r_ksztaltna(X, A, B, Val) :-
    ( X =< A -> Val = 0.0
    ; X > A, X =< B -> Val is (X - A) / (B - A)
    ; Val = 1.0
    ).
```

2.1.2 Atrybuty rozmyte w systemie

W tabeli 2 przedstawiono wszystkie atrybuty rozmyte wraz z przypisanymi funkcjami przynależności (zgodnie z definicjami w `fuzzy.pl`).

Tabela 2: Wykaz atrybutów rozmytych używanych w systemie

Atrybut	Wartość	Funkcja przynależności	Opis
Ciężar	bardzo lekki	L-kształtna(0.5, 1)	Orientacyjna waga zwierzęcia w kg.
	lekki	Trapez(0.5, 5, 15, 25)	
	średni	Trapez(15, 20, 50, 100)	
	ciężki	Trapez(50, 100, 400, 500)	
	bardzo ciężki	R-kształtna(500, 1000)	
Długość	bardzo mały	L-kształtna(10, 20)	Orientacyjna długość ciała w cm.
	mały	Trapez(10, 15, 35, 50)	
	średni	Trapez(20, 30, 50, 120)	
	duży	Trapez(80, 200, 400, 500)	
	bardzo duży	R-kształtna(400, 600)	
Długość życia	krótka	L-kształtna(5, 15)	Typowa długość życia w latach.
	średnia	Trapez(10, 15, 20, 30)	
	długa	R-kształtna(20, 30)	

Na podstawie tych atrybutów generowany jest zbiorczy atrybut **Wielkość**, wykorzystywany następnie w module wnioskowania przybliżonego. Atrybut ten przyjmuje wartości: *bardzo mała*, *mała*, *średnia*, *duża*, *bardzo duża* oraz *niestandardowa*.

2.2 Pozostałe atrybuty

Wiele cech charakteryzujących zwierzęta ma charakter kategoriowy – przypisuje się im jedną konkretną wartość. Dla tych cech stosujemy atrybuty nierozmyte. Ich wykaz znajduje się w tabeli 3.

2.3 Implementacja bazy wiedzy

Baza wiedzy zawiera **100 gatunków** zwierząt. Aby uniknąć ręcznego wpisywania faktów, dane przygotowano w pliku `zwierzeta_atrybuty.csv`, a plik `kb.pl` generowany jest automatycznie skryptem `gen_kb.py`. Przykładowy fragment wygenerowanej bazy:

Listing 2: Fragment bazy wiedzy (`kb.pl`)

```
animal('lew').
attribute('lew', dieta, 'miesozerne').
attribute('lew', srodowisko, 'ladowe').
attribute('lew', skrzydla, 'nie').
attribute('lew', nogi, 'cztery').
attribute('lew', gromada, 'ssaki').
attribute('lew', kontynent, 'afryka').
% ... pozostale atrybuty dyskretne ...
continuous_attribute('lew', ciezar, 190.0).
continuous_attribute('lew', dlugosc, 250.0).
continuous_attribute('lew', dlugosc_zycia, 12.0).
```

Baza udostępnia tylko fakty – nie zawiera żadnej logiki sterującej ani statycznie zapisanych konkluzji, co realizuje wymóg separacji warstw (rozdział 6).

Tabela 3: Wykaz atrybutów dyskretnych używanych w systemie

Atrybut	Wartości	Opis
Dieta	roślinożerne, wszystkożerne, mięsożerne	Główny rodzaj spożywanego pokarmu.
Środowisko życia	lądowe, wodne, powietrzne	Środowisko typowe dla gatunku.
Skrzydła?	tak, nie	Czy zwierzę posiada skrzydła.
Liczba nóg/odnóży	dwa, cztery, sześć, osiem, więcej, brak	Liczba nóg lub odnóży.
Udomowione?	tak, nie	Czy zwierzę jest udomowione.
Strategia rozrodcza	żyworodne, jajorodne, jajożyworodne	Sposób wydawania potomstwa.
Futro?	tak, nie	Czy zwierzę posiada futro.
Typ klimatu	chłodny, umiarkowany, ciepły	Typ klimatu występowania.
Gromada	płazy, gady, ryby, ptaki, ssaki, inne	Przynależność do gromady.
Stadne?	tak, nie	Czy zwierzę ma instynkt stadny.
Kontynent	Azja, Afryka, Ameryka Płn., Ameryka Płd., Europa, Antarktyda, Australia	Kontynent występowania.

3 Mechanizm Wnioskowania i Interakcja z Użytkownikiem

System nie realizuje z góry ustalonej, statycznej sekwencji pytań. Przebieg dialogu jest dynamicznym wynikiem procesu poszukiwania rozwiązania. Logika dialogu zaimplementowana jest w pliku `ui.pl`, a sterowana jest pętlą `ask_loop/2`, która utrzymuje aktualny zbiór możliwych hipotez (*PossibleAnimals*) oraz zbiór jeszcze niewykorzystanych atrybutów.

3.1 Dynamiczna selekcja pytań

Interfejs na bieżąco analizuje aktualną przestrzeń hipotez i wybiera do zadania pytanie o atrybut o **najwyższej mocy dyskryminacyjnej**. Moc ta jest mierzona za pomocą **entropii** rozkładu wartości atrybutu wśród rozważanych zwierząt – wybierany jest atrybut o najniższej entropii warunkowej, czyli ten, który najsilniej zawęży zbiór hipotez. Atrybuty o jednej wartości (bezużyteczne dla rozróżnienia) są pomijane.

$$H(a) = - \sum_{v \in V_a} p(v) \log p(v) \quad (3)$$

Listing 3: Selekcja atrybutu na podstawie entropii (`ui.pl`)

```
best_attribute_to_ask(Attrs, Animals, BestAttr) :-
    findall(Score-Attr, (
        member(Attr, Attrs),
        evaluate_attribute(Attr, Animals, Score)
    ), Scores),
    ( Scores = [] -> BestAttr = none
    ; sort(1, @=<, Scores, [MinScore-BestAttr|_]), % minimalizujemy entropie
      MinScore < 999 -> true
    ; BestAttr = none
    ).
```

3.2 Kontekstowość dialogu

Zgromadzone dotychczas informacje są zawsze uwzględniane przy wyborze kolejnego pytania – po każdej odpowiedzi zbiór hipotez jest filtrowany (`filter_animals/4`), a kolejne pytanie dobierane jest już do nowej, zawężonej przestrzeni. Dzięki temu pytania o atrybuty, które straciły moc rozróżniającą, nie są zadawane.

3.3 Wielokrotny wybór

System dopuszcza odpowiedzi wielokrotnego wyboru wszędzie tam, gdzie wynika to z natury pytania. Użytkownik może podać kilka wartości oddzielonych przecinkami (np. `tak,nie`); są one wówczas obsługiwane przez `filter_animals_multi/4`, który zachowuje zwierzęta pasujące do *którekolwiek* ze wskazanych wartości.

4 Moduł wnioskowania rozmytego

Pierwszym etapem pracy systemu jest wnioskowanie rozmyte. Moduł operuje na zbiorach rozmytych stworzonych na podstawie atrybutów *ciężar*, *długość* i *długość życia*. Na ich podstawie tworzy nowy, zbiorczy atrybut *wielkość*. Wnioskowanie oparte jest na iloczynowej t-normie.

Proces składa się z trzech etapów:

1. **Rozmywanie** – wartość liczbowa konwertowana jest na stopnie przynależności do zbiorów rozmytych za pomocą funkcji z tabeli 2.
2. **Wnioskowanie** – reguły rozmyte łączą przynależności poszczególnych cech. Przyjęta iloczynowa t-norma indukuje operatory:

$$a \wedge b = \top(a, b) = a \cdot b \quad (4)$$

$$a \vee b = \perp(a, b) = a + b - a \cdot b \quad (5)$$

W implementacji koniunkcja realizowana jest przez predykat `t_norma/3`, a alternatywa przez `s_norma/3`.

3. **Wyostrażanie** – wartość wynikowa wybierana jest metodą maksimum (reguła rozmyta o najwyższym stopniu spełnienia wyznacza wartość atrybutu *wielkość*):

$$\arg \max_{x \in U} \mu(x) \quad (6)$$

Listing 4: Reguła rozmyta i defuzyfikacja (`fuzzy.pl`)

```
t_norma(A, B, V) :- V is A * B.
s_norma(A, B, V) :- V is A + B - (A * B).

fuzzy_rule(srednia, Ciezar, Dlugosc, _Zycie, V) :-
    ciezar(Ciezar, sredni, Vc),
    dlugosc(Dlugosc, sredni, Vd),
    t_norma(Vc, Vd, V).

calculate_wielkosc(Ciezar, Dlugosc, Zycie, Wielkosc) :-
    findall(V-W, fuzzy_rule(W, Ciezar, Dlugosc, Zycie, V), Results),
    sort(0, @>=, Results, [MaxV-MaxW | _]),
    ( MaxV > 0 -> Wielkosc = MaxW
    ; Wielkosc = niestandardowa
    ).
```

Uwaga implementacyjna: reguły rozmyte zostały przyjęte w uproszczonej postaci łączącej cechy *ciężar* i *długość* (zgodnie z komentarzem w kodzie), ponieważ pełny zestaw reguł nie był jednoznacznie określony w specyfikacji. Jeżeli żadna reguła nie zostaje spełniona w stopniu większym od zera, atrybutowi nadawana jest wartość *niestandardowa*.

4.1 Przykład liczbowy

Prześledźmy działanie modułu na przykładzie *kozy* o parametrach: ciężar = 60 kg, długość = 110 cm, długość życia = 15 lat.

Krok 1 – rozmywanie. Wartości liczbowe przekładane są na stopnie przynależności do zbiorów rozmytych (tabela 2). Niezerowe przynależności wynoszą:

$$\begin{aligned} \text{ciężar } 60 : \mu_{\text{średni}} &= \frac{100-60}{100-50} = 0,80, & \mu_{\text{ciężki}} &= \frac{60-50}{100-50} = 0,20, \\ \text{długość } 110 : \mu_{\text{średni}} &= \frac{120-110}{120-50} \approx 0,14, & \mu_{\text{duży}} &= \frac{110-80}{200-80} = 0,25. \end{aligned}$$

Krok 2 – wnioskowanie. Reguły rozmyte łączą przynależności iloczynową t-normą ($\top(a, b) = a \cdot b$). Spełnienie uzyskują dwie reguły:

$$\begin{aligned} \text{wielkość} = \text{średnia} : \mu_{\text{średni}}^{\text{ciężar}} \cdot \mu_{\text{średni}}^{\text{długość}} &= 0,80 \cdot 0,14 \approx \mathbf{0,114}, \\ \text{wielkość} = \text{duża} : \mu_{\text{ciężki}}^{\text{ciężar}} \cdot \mu_{\text{duży}}^{\text{długość}} &= 0,20 \cdot 0,25 = 0,050. \end{aligned}$$

Krok 3 – wyostrzanie. Metodą maksimum wybierana jest reguła o najwyższym stopniu spełnienia:

$$\arg \max\{0,114; 0,050\} \implies \text{wielkość} = \text{średnia}.$$

Wynik ten został potwierdzony uruchomieniem predykatu `calculate_wielkosc(60.0, 110.0, 15.0, W)`, który zwraca `W = srednia`.

5 Moduł wnioskowania przybliżonego

Moduł wnioskowania przybliżonego (`rough.pl`) przeprowadza ostateczną klasyfikację zwierzęcia. Na wejściu otrzymuje atrybuty warunkowe pochodzące od użytkownika oraz jeden atrybut (*wielkość*) wyznaczony przez moduł wnioskowania rozmytego. Matematyczną podstawą modułu jest teoria zbiorów przybliżonych.

Tablica decyzyjna to para $DT = (U, A \cup \{d\})$, gdzie U to zbiór obiektów (gatunków), A to zbiór atrybutów warunkowych, a d jest atrybutem decyzyjnym. Zbiór atrybutów warunkowych zdefiniowany jest jako:

```
all_attributes([wielkosc, dieta, srodowisko, skrzydla, nogi, udomowione,
               rozrod, futro, klimat, gromada, stadne, kontynent]).
```

Wartości atrybutów pobierane są jednolicie przez predykat `get_attr/3`, który dla atrybutu *wielkość* sięga do wyniku modułu rozmytego, a dla pozostałych – do bazy wiedzy. Przykładowy fragment tablicy decyzyjnej (dla czytelności ograniczony do podzbioru atrybutów warunkowych oraz sześciu obiektów) przedstawiono w tabeli 4. Wartości atrybutu *wielkość* pochodzą z rzeczywistych obliczeń modułu rozmytego.

Zwróćmy uwagę, że obiekty 1 i 3 (lew, krokodyl) mają identyczne wartości w przedstawionym podzbiorze atrybutów – do ich rozróżnienia konieczne są dodatkowe atrybuty (np. *gromada*, *środowisko*), co ilustruje rolę pełnego zbioru atrybutów warunkowych.

Tabela 4: Tablica decyzyjna – klasyfikacja zwierząt (fragment)

Obiekt	Wielkość	Dieta	Skrzydła?	Nogi	Gatunek (<i>d</i>)
1	duża	mięsożerne	nie	cztery	lew
2	bardzo duża	roślinożerne	nie	cztery	słoń
3	duża	mięsożerne	nie	cztery	krokodyl
4	bardzo mała	roślinożerne	tak	sześć	pszczoła
5	niestandard.	mięsożerne	tak	dwa	orzeł
6	bardzo mała	roślinożerne	nie	cztery	mysz

5.1 Relacja nierozróżnialności

Relację nierozróżnialności generowaną przez zbiór atrybutów $B \subseteq A$ definiujemy następująco:

$$\text{IND}(B) = \{(x, y) \in U \times U : \forall_{a \in B} a(x) = a(y)\}$$

Jest to relacja równoważności dzieląca U na klasy abstrakcji obiektów nierozróżnialnych. Klasa abstrakcji obiektu x :

$$[x]_B = \{y \in U : (x, y) \in \text{IND}(B)\}$$

W implementacji odpowiadają im predykaty `indiscernible/3`, `equivalence_class/4` oraz `all_equivalence_c`.

5.2 Przybliżenie dolne i górne

Dla zbioru $X \subseteq U$ względem $B \subseteq A$ definiujemy:

$$\underline{B}(X) = \bigcup \{Y \in U/\text{IND}(B) : Y \subseteq X\}, \quad \overline{B}(X) = \bigcup \{Y \in U/\text{IND}(B) : Y \cap X \neq \emptyset\}$$

Realizują je predykaty `lower_approximation/3` i `upper_approximation/3`.

5.3 Niespójność

Niespójność zachodzi, gdy dwa obiekty nierozróżnialne przez A mają różne decyzje. Zaproponowany system nie dopuszcza niespójności – są one usuwane **metodą jakościową**. Dla każdego obiektu wyznaczana jest dokładność przybliżenia dolnego:

$$\gamma_B(X) = \frac{|\underline{B}(X)|}{|U|}$$

a usuwany jest obiekt o najniższej dokładności. Procedurę realizuje predykat `remove_inconsistencies/0` wraz z `find_worst_object/4`.

5.4 Atrybuty niezbędne i jądro systemu

Atrybut $a \in B$ nazywamy **zbędnym**, jeśli $\text{IND}(B) = \text{IND}(B \setminus \{a\})$; w przeciwnym razie jest **niezbędny**. Zbiór wszystkich atrybutów niezbędnych nazywamy jądrem $\text{CORE}(B)$. W systemie jądro wyznaczane jest dwiema metodami, które służą do wzajemnej weryfikacji:

- jako zbiór atrybutów występujących pojedynczo we wpisach macierzy nierozróżnialności (`core_from_matrix/2`),
- przez bezpośrednie sprawdzenie warunku $\text{IND}(B) \neq \text{IND}(B \setminus \{a\})$ (`core_attributes_verify/3`).

5.5 Redukty

Redukt to minimalny zbiór atrybutów zachowujący rozróżnialność obiektów – musi zawierać jądro oraz mieć niepuste przecięcie z każdym niepustym wpisem macierzy nierozróżnialności.

5.5.1 Wyznaczanie reduktów z macierzy nierozróżnialności

Macierz nierozróżnialności o wymiarach $n \times n$ ($n = |U|$) ma elementy:

$$M_{ij} = \{a \in A : a(x_i) \neq a(x_j)\}$$

Redukty wyznaczone są metodą **zachłanną**: algorytm startuje od jądra, dodaje atrybuty pokrywające najwięcej niepokrytych wpisów (**greedy_cover/4**), a następnie próbuje usunąć atrybuty zbędne (**minimize_reduct/4**), tak by uzyskać zbiór minimalny.

5.6 Reguły minimalne

Przez regułę decyzyjną rozumiemy implikację postaci:

$$a_1(x) = v_1 \wedge a_2(x) = v_2 \wedge \dots \wedge a_k(x) = v_k \implies dec(x) = d$$

Generowanie reguł minimalnych przeprowadzono zgodnie z algorytmem zaproponowanym przez Z. Pawlaka i A. Skowrona (*A rough set approach to decision rules generation*):

1. usunięcie niespójności z tablicy decyzyjnej *DT* (metodą jakościową),
2. utworzenie macierzy nierozróżnialności,
3. dla każdego obiektu: utworzenie uogólnionej macierzy nierozróżnialności względem decyzji, jej minimalizacja (wyznaczenie reduktu) i zapis reguły decyzyjnej.

Całość koordynuje predykat **generate_reducts_and_rules/0**, a poszczególne reguły zapisywane są dynamicznie jako fakty **dec_rule/3**. Klasyfikacja końcowa (**get_decision/2**) odbywa się przez dopasowanie reguł do faktów podanych przez użytkownika i wybór decyzji o największej liczbie pasujących reguł.

Listing 5: Generowanie reguły dla obiektu (**rough.pl**)

```
generate_rules_for_animal(Animals, Attrs, Animal) :-
    decision_discernibility_matrix(Animal, Animals, Attrs, Matrix),
    ( Matrix = [] -> true
    ; find_reduct(Matrix, Attrs, Reduct),
      create_rule(Animal, Reduct)
    ).
```

5.7 Przykład: wyznaczenie reguły minimalnej

Prześledźmy algorytm na uproszczonej tablicy decyzyjnej (tabela 5), ograniczonej do czterech atrybutów warunkowych $A = \{gromada, \text{środowisko}, \text{skrzydła}, \text{dieta}\}$.

Tabela 5: Uproszczona tablica decyzyjna dla przykładu

Obiekt	Gromada	Środowisko	Skrzydła?	Dieta	<i>d</i>
x_1	ssaki	lądowe	nie	mięsożerne	lew
x_2	ptaki	powietrzne	tak	mięsożerne	orzeł
x_3	gady	wodne	nie	mięsożerne	krokodyl
x_4	ssaki	lądowe	nie	roślinożerne	słoń
x_5	płazy	wodne	nie	mięsożerne	żaba

Macierz nierozróżnialności względem decyzji dla x_1 . Dla obiektu x_1 (lew) tworzymy wpisy $M_{1,j} = \{a \in A : a(x_1) \neq a(x_j)\}$ porównując go z obiektami o innej decyzji:

$$\begin{aligned} M_{1,2} &= \{gromada, \acute{s}rodowisko, skrzydła\}, & M_{1,3} &= \{gromada, \acute{s}rodowisko\}, \\ M_{1,4} &= \{dieta\}, & M_{1,5} &= \{gromada, \acute{s}rodowisko\}. \end{aligned}$$

Funkcja nierozróżnialności i jej minimalizacja. Funkcja przyjmuje postać koniunkcji alternatyw (CNF):

$$f(x_1) = (gr \vee \acute{s}r \vee sk) \wedge (gr \vee \acute{s}r) \wedge (dieta) \wedge (gr \vee \acute{s}r).$$

Atrybut *dieta* występuje samodzielnie – należy do jądra i musi znaleźć się w redukcji. Pozostałe klauzule pokrywa pojedynczy atrybut *gromada* (lub równoważnie *środowisko*). Algorytm zachłanny (`find_reduct/3`) wybiera zatem redukt $\{dieta, gromada\}$.

Zapis reguły. Na podstawie reduktu i wartości atrybutów obiektu x_1 powstaje reguła:

$$gromada = ssaki \wedge dieta = \text{mięsożerne} \implies lew.$$

W pełnym systemie (12 atrybutów warunkowych) dla lwa generowana jest analogiczna, lecz dłuższa reguła oparta na innych atrybutach rozróżniających: `dieta=mięsożerne` $\wedge$ `kontynent=afryka` $\wedge$ `stadne=tak` $\implies$ `lew`.

6 Architektura Systemu – Separacja Warstw

Projekt przestrzega zasady ścisłego rozdzielania warstw, co jest kryterium projektowym o szczególnym znaczeniu dla systemów z bazą wiedzy.

6.1 Komponenty systemu

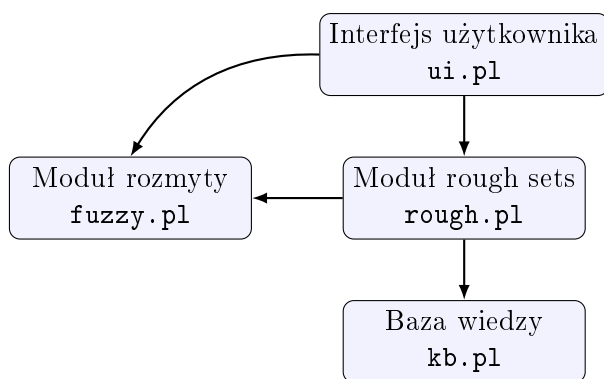
System podzielony jest na cztery główne komponenty, każdy w osobnym module Prologa:

1. **Baza danych i wiedzy** (`kb.pl`) – zawiera wyłącznie fakty i reguły dziedzinowe (zoologia). Nie zawiera logiki sterującej ani elementów interfejsu.
2. **Mechanizm wnioskowania rozmytego** (`fuzzy.pl`) – implementuje czystą logikę rozmytą: funkcje przynależności, operatory t- i s-normy oraz wyznaczanie atrybutu *wielkość*. Nie zna interfejsu użytkownika.
3. **Moduł znajdowania reguł minimalnych** (`rough.pl`) – oddzielny komponent umożliwiający wydrukowanie atrybutów niezbędnych i reguł minimalnych na potrzeby analizy bazy wiedzy (`print_indispensable/0`, `find_minimal_rules/0`).
4. **Interfejs użytkownika** (`ui.pl`) – odpowiada wyłącznie za komunikację z użytkownikiem i formatowanie wyników. Nie zawiera wiedzy dziedzinowej.

6.2 Wymagania dotyczące separacji

Separacja warstw oznacza w szczególności:

- dane przedmiotowe nie pojawiają się w warstwie interfejsu,
- baza wiedzy jest oddzielona zarówno od interfejsu, jak i od logiki sterowania,
- konkluzje (reguły decyzyjne) konstruowane są dynamicznie w trakcie procesu wnioskowania (`dec_rule/3` jako fakty dynamiczne) – nie są reprezentowane jako statyczne fakty w bazie wiedzy.



Rysunek 1: Zależności między modułami systemu

7 Instalacja i uruchomienie

Do uruchomienia systemu wymagany jest interpreter **SWI-Prolog** (<https://www.swi-prolog.org>). System startuje poleceniem wydanym w katalogu głównym projektu:

```
swipl main.pl
```

Po starcie wyświetlane jest menu z dwiema opcjami:

1. **Uruchom system ekspertowy** – interaktywna identyfikacja gatunku (dialog z dynamiczną selekcją pytań),
2. **Analiza bazy wiedzy** – wypisanie atrybutów niezbędnych (jądra) oraz wygenerowanych reguł minimalnych.

7.1 Rozszerzanie bazy wiedzy

Baza wiedzy generowana jest automatycznie z pliku `zwierzeta_atrybuty.csv`. Aby dodać nowy gatunek, wystarczy dopisać wiersz do pliku CSV (kolumny: *Zwierze*, *dieta*, *srodowisko*, *skrzydla*, *nogi*, *udomowione*, *rozrod*, *futro*, *klimat*, *gromada*, *stadne*, *kontynent*, *ciezar*, *dlugosc*, *zycie*) i ponownie wygenerować bazę:

```
python gen_kb.py # tworzy na nowo src/kb.pl
```

Atrybuty ciągłe (*ciezar*, *dlugosc*, *zycie*) podaje się jako liczby; pozostałe – jako wartości tekstowe zgodne z tabelami 2 i 3.

7.2 Analiza bazy wiedzy

Wybór opcji 2 uruchamia moduł analizy. Dla aktualnej bazy (100 gatunków) moduł zwraca jądro systemu oraz zbiór reguł minimalnych:

```

Atrybuty niezbedne (Core) w calym systemie:
[dieta, futro, gromada, kontynent, srodowisko, stadne, udomowione, wielkosc]

Reguly minimalne wygenerowane pomyslnie.
Liczba regul: 91
Przykladowe reguly:
dieta=miesozerne ^ kontynent=afryka ^ stadne=tak => lew
kontynent=azja ^ wielkosc=duza => tygrys
klimat=umiarkowany ^ stadne=nie ^ wielkosc=duza => niedzwiedz
  
```

Jądro liczy 8 atrybutów – są to cechy, których usunięcie naruszyłoby rozróżnialność gatunków. Atrybuty *skrzydła*, *nogi*, *rozród* i *klimat* okazały się w tym zbiorze zbędne (redundantne), co nie znaczy, że nie pojawiają się w pojedynczych regułach minimalnych.

8 Przykładowa sesja identyfikacji

Poniżej przedstawiono rzeczywisty zapis sesji identyfikacji *lewa*. System inicjalizuje bazę, generuje reguły, a następnie zadaje pytania o atrybutach o najwyższej mocy dyskryminacyjnej, zawężając zbiór hipotez po każdej odpowiedzi. Odpowiedzi użytkownika oznaczono symbolem „>”.

```
Wybierz opcje:
1. Uruchom system ekspertowy (identyfikacja zwierzęcia)
2. Analiza bazy wiedzy (reguły minimalne i atrybuty niezbędne)
Twój wybór (1/2): > 1
-----
System Ekspertowy - Identyfikacja Zwierząt
-----
Inicjalizacja bazy wiedzy i wnioskowanie rough sets...
Gotowe. Rozpoczynamy dialog.
Pytanie: Czy to zwierze jest udomowione?
Mozliwe opcje: [nie,tak] > nie
Pytanie: Czy to zwierze posiada skrzydła?
Mozliwe opcje: [nie,tak] > nie
Pytanie: W jakim srodowisku zyje to zwierze?
Mozliwe opcje: [ladowe,wodne] > ladowe
Pytanie: Ile nog/odnozy ma to zwierze?
Mozliwe opcje: [cztery,brak,dwa,osiem] > cztery
Pytanie: Jaka jest strategia rozrodcza zwierzęcia?
Mozliwe opcje: [zyworodne,jajorodne] > zyworodne
Pytanie: Czy to zwierze posiada futro?
Mozliwe opcje: [tak,nie] > tak
Pytanie: Czy to zwierze jest stadne?
Mozliwe opcje: [tak,nie] > tak
Pytanie: W jakim klimacie najczesciej zyje to zwierze?
Mozliwe opcje: [cieply,umiarkowany] > cieply
Pytanie: Z jakiego kontynentu pochodzi to zwierze?
Mozliwe opcje: [afryka,australia] > afryka
Pytanie: Jaka jest glowna dieta zwierzęcia?
Mozliwe opcje: [miesozerne,roslinozerne] > miesozerne
-----
Zidentyfikowane zwierze to: lew
Decyzja z modulu Rough Set: lew
-----
```

W tej sesji do jednoznacznej identyfikacji wystarczyło 10 pytań (na 12 możliwych atrybutów). Gdyby w grę wchodziło rozróżnienie po wielkości, system poprosiłby dodatkowo o liczbowy ciężar, długość i długość życia, po czym sam wyznaczyłby kategorię wielkości metodą wnioskowania rozmytego (rozdział 4).

9 Testy i Weryfikacja

Poprawność systemu weryfikowano na kilku poziomach:

- **Moduł rozmyty** – wartości funkcji przynależności oraz wynik atrybutu *wielkość* sprawdzano dla wybranych gatunków przez bezpośrednie wywołanie `calculate_wielkosc/4` (np. koza → *średnia*, lew → *duża*, słoń → *bardzo duża*).
- **Spójność bazy wiedzy** – moduł analizy wykrywa i usuwa niespójności metodą jakościową; po tym etapie z 91 wygenerowanych reguł żadna nie prowadzi do sprzecznej decyzji.
- **Jądro systemu** – wyznaczane jest dwiema niezależnymi metodami (z macierzy nierozróżnialności oraz przez warunek $IND(B) \neq IND(B \setminus \{a\})$), które dają zgodny wynik, co potwierdza poprawność implementacji.

- **Dialog** – przeprowadzono sesje identyfikacyjne dla gatunków z różnych gromad, weryfikując, że system zbiega do jednego gatunku oraz że decyzja modułu rough set pokrywa się z wynikiem dialogu.

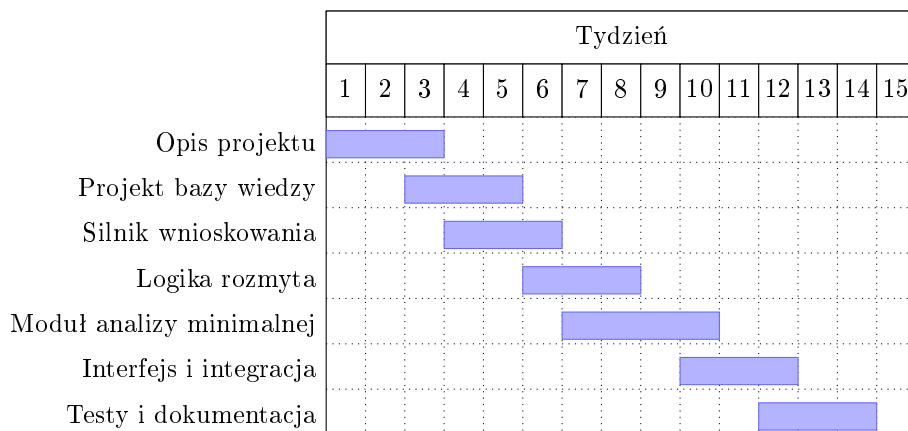
10 Ograniczenia i Kierunki Rozwoju

Zrealizowany system ma kilka znanych ograniczeń, które wyznaczają naturalne kierunki dalszego rozwoju:

- **Uprozczone reguły rozmyte** – atrybut *wielkość* wyznaczany jest na podstawie sparowanych kategorii ciężaru i długości; dla nietypowych proporcji (np. długi, lecz lekki wąż) zwracana jest wartość *niestandardowa*. Rozszerzenie zbioru reguł rozmytych poprawiłoby pokrycie takich przypadków.
- **Brak obsługi błędnych odpowiedzi** – podanie wartości spoza listy opcji zawęży zbiór hipotez do pustego; warto dodać walidację i możliwość cofnięcia odpowiedzi.
- **Wagi reguł** – obecnie wszystkie reguły mają wagę 1 ($\text{dec_rule}/3$); wprowadzenie wag opartych na pokryciu lub pewności umożliwiłoby ranking konkurencyjnych hipotez.
- **Interfejs** – system działa w trybie tekstowym; możliwe jest opracowanie graficznego interfejsu użytkownika.

11 Harmonogram Realizacji

Prace nad projektem prowadzone były w cyklu 15-tygodniowym. Faktyczny przebieg realizacji poszczególnych etapów przedstawia diagram Gantta (rysunek 2).



Rysunek 2: Diagram Gantta – harmonogram realizacji projektu

12 Podział Obowiązków

Realizacja projektu została podzielona pomiędzy członków zespołu zgodnie z tabelą 6. Każdy z autorów odpowiadał za wydzielony moduł systemu, przy wspólnej pracy nad integracją i dokumentacją.

13 Podsumowanie

Zrealizowany system ekspertowy do identyfikacji gatunków zwierząt stanowi praktyczną implementację koncepcji wnioskowania regułowego wzbogaconego o mechanizmy logiki rozmytej i teorii zbiorów przybliżonych. Kluczowymi elementami wyróżniającymi system są:

Tabela 6: Podział obowiązków w zespole projektowym

Osoba	Zakres odpowiedzialności
Krzysztof Gryboś	Baza wiedzy (<code>kb.pl</code>), zbiór danych <code>zwierzeta_atrybuty.csv</code> oraz skrypt generujący <code>gen_kb.py</code> .
Michał Piechota	Moduł wnioskowania rozmytego (<code>fuzzy.pl</code>) – funkcje przynależności, operatory t-/s-normy, wyznaczanie atrybutu <i>wielkość</i> .
Sebastian Rydz	Moduł wnioskowania przybliżonego (<code>rough.pl</code>) – relacja nierozróżnialności, redukty, reguły minimalne, klasyfikacja.
Hubert Sobociński	Interfejs użytkownika (<code>ui.pl</code>), dynamiczna selekcja pytań, integracja modułów i dokumentacja.

- **Wnioskowanie w warunkach niepewności** – dzięki funkcjom przynależności system radzi sobie z rozmytymi parametrami i płynnymi granicami między kategoriami,
- **Dynamiczny, kontekstowy dialog** – liczba pytań jest minimalizowana przez wybór atrybutów o najniższej entropii,
- **Moduł analizy minimalnej** – narzędzie do weryfikacji i optymalizacji bazy wiedzy, wykrywające niespójności, atrybuty niezbędne i redukty,
- **Czysta separacja warstw** – architektura gwarantująca niezależność bazy wiedzy, silnika wnioskowania, modułu analizy i interfejsu.

Zastosowanie języka Prolog jako platformy implementacyjnej pozwoliło w naturalny sposób wyrazić reguły produkcji i mechanizm wnioskowania wstecznego, a rozszerzenie o operatory rozmyte i zbiory przybliżone umożliwiło obsługę rzeczywistych, niedoskonałych danych wejściowych.

Literatura

- [1] Z. Pawlak, *Rough sets*, International Journal of Computer and Information Sciences, 11(5):341–356, 1982.
- [2] Z. Pawlak, A. Skowron, *Rough sets and Boolean reasoning*, Information Sciences, 177(1):41–73, 2007.
- [3] A. Skowron, C. Rauszer, *The discernibility matrices and functions in information systems*, w: Intelligent Decision Support, Springer, s. 331–362, 1992.
- [4] L. A. Zadeh, *Fuzzy sets*, Information and Control, 8(3):338–353, 1965.
- [5] J. Wielemaker i in., *SWI-Prolog Reference Manual*, <https://www.swi-prolog.org>.